POLYTECHNICS MAURITIUS LTD

DAD2104 — Analysis & Design in Practice

PROJECT REPORT

Smart Pill Reminder and Dispenser

(Project name: Omni-Care)

Diploma in Emerging Technologies (IoT / Big Data Analytics)

Field	Details
Group Number	Group 13
Group Members	Reelesh Ganesh Bhoyrub (2025_20565_7005) — IoT Theerouven Chinnecunnen (2025_20864_6989) — BDA Yashveer Boodhna (2025_20594_6993) — BDA
Programme	Diploma in Emerging Technologies — IoT / BDA
Cohort	DET IoT/BDA C7
Lecturer	Zoubeir Aungnoo
Submission Date	22/ 07/ 2026
Module Code	DAD2104

Declaration of Originality

We, the members of Group 13, confirm that:

1. This report is our own work, and any information we used from other sources has been referenced.
2. We used the Harvard referencing style for all sources.
3. This report has not been submitted for any other module or course.
4. We understand that copying other people's work without credit is a serious offence.

Name	Student ID	Contribution	Signature
Reelesh Ganesh Bhoyrub	2025_20565_7005	Hardware wiring, firmware coding, Wokwi simulation, Blynk app integration, report writing	
Theerouven Chinnecunnen	2025_20864_6989	Surveys and questionnaires	
Yashveer Boodhna	2025_20594_6993	PowerPoint Presentation	

Link to project files (OneDrive/Wokwi): https://wokwi.com/projects/470102751638646785

Abstract

People with Alzheimer's disease often forget if they have taken their medicine. This can lead to missed doses or taking medicine twice, which is dangerous. This project, called the Smart Pill Reminder and Dispenser (Omni-Care), is a device that solves this problem. It stores pills in tubes, and at the right time, a small motor (servo) opens the tube to release the pill. A button must be pressed to confirm the pill was taken, otherwise an alarm keeps sounding.

The device uses an ESP32 microcontroller, a real-time clock (RTC) to keep track of time, an LCD screen to show the time and messages, a buzzer and LEDs for alerts, and two servo motors for two pill tubes. We also connected the device to the internet using Wi-Fi and the Blynk app, so a caregiver can change the pill times remotely and see the status of the device from their phone, without needing to reprogram the device.

We built and tested the full project as a working simulation on Wokwi, an online circuit simulator. We also collected feedback from a nurse, elderly people and a pregnant woman through short surveys, to understand what they need from a device like this. This report explains the problem, our research, our design, and how we tested the system.

Table of Contents

List of Figures

List of Tables

Acronyms & Glossary

Term	Meaning
IoT	Internet of Things
RTC	Real-Time Clock — keeps track of the current time and date
LCD	Liquid Crystal Display
MCU	Microcontroller Unit — the "brain" of the device (here, the ESP32)
Blynk	A cloud platform and mobile app used to control and monitor IoT devices
Virtual Pin	A software channel used by Blynk to send/receive data between the app and the device
DFD	Data Flow Diagram
ERD	Entity-Relationship Diagram — shows how data is organised
UML	Unified Modeling Language
FR / NFR	Functional Requirement / Non-Functional Requirement
BOM	Bill of Materials — list of parts needed
I2C	A two-wire connection used to link small chips like the LCD and RTC

1. Introduction

The Internet of Things (IoT) has grown from a new and emerging technology into a practical solution for creating affordable smart systems that can sense, respond and communicate with other devices. One area where the impact of IoT can be witnessed is the healthcare. By combining sensors and internet connectivity with everyday medical devices have shown to be very effective and help to improve patient safety and make healthcare more efficient.

Many elderly people, especially those with Alzheimer's disease, find it hard to remember if they have taken their medicine. This can cause serious health problems. At the same time, Internet of Things (IoT) technology has become cheap and easy to use, which means students can now build smart, connected devices to help solve real-world problems like this one.

This project combines concepts from the Internet of Things, such as sensors , automation and connected devices, with basic data management techniques used to record medication schedules and adherence history. As a result, it brings together knowledge from both IoT and Big Data Analytics, making it well suited to the objectives of the DAD2104.

This report explains our project: the Smart Pill Reminder and Dispenser (also called Omni-Care). It is a device that automatically gives out pills at the right time, checks that the pill was taken, and can be controlled remotely through a mobile app. This report follows the structure required for the DAD2104 module. Part A covers the analysis of the problem and our plan. Part B covers the design of the system. Part C covers how we built it. Part D covers how we tested it.

2. Problem Statement

People with Alzheimer's disease slowly lose their short-term memory. This makes it hard for them to remember if they already took their medicine. As a result, they might skip a dose, take it twice, or take the wrong medicine. This can make their health worse and sometimes send them to hospital.

With research that I have conducted, I have found that we live in an aging global trend. Where there are more elders than youngster. So the population affected by Alzheimer is not shrinking but keep growing. This is why it is important to find a solution to this problem.

Right now, most people use simple pillboxes or phone alarms. These only remind the person — they do not check if the medicine was actually taken. On the other hand, some companies sell smart pill dispensers that do check this, but they are expensive and usually need a paid subscription.

Our project aims to bridge that gap by developing a low-cost IoT based smart medicine dispenser that is both practical and affordable. The system is design in a way to give caretaker reliable confirmation that medication has been taken while allowing patients to maintain their independence. By combining automatic pill dispensing, confirmation through user interaction and remote monitoring, the proposed solution provides many of the key benefits of commercial systems without the high cost or ongoing subscription fees. The goal is to create a solution that is completely accessible easy to understand and suitable for patient suffering from alzheimer or any kind of needs for this our project.

3. Literature Review

3.1 Background

Taking medicine correctly and on time is called "medication adherence". It refers to how well a patient follows the instructions given by a health care professional when taking medication. Studies show that people with memory problems often struggle with this, and it can lead to the deterioration of their own health (World Health Organization, 2003). Since the problem is mainly about memory, not willingness, a good solution should not depend on the patient remembering anything by themselves as sometimes their brain or memory system doesn't work like it should.

This project is based on the IoT, where physical devices can collect information, communicate through a network and perform actions automatically. The proposed system follows the three main layers of IoT architecture which are:

1. The Perception layer (Sensors, Actuators).
2. The Network layer Which allow communication between devices.
3. The Application layr where users can monitor and manage the system.

3.2 Existing Solutions

There are three common types of solutions available today:

- Simple pillboxes and phone reminders — cheap, but they cannot check if the pill was actually taken.
- Locked pillboxes — these only open at the right time, but still cannot confirm the pill was removed and taken.
- Smart connected dispensers (like Hero or MedMinder) — these check adherence and alert a caregiver, but they are expensive and usually need a monthly subscription.

3.3 Comparative Study

Solution	Checks if pill was taken?	Alerts a caregiver?	Cost
Simple pillbox / phone reminder	No	No	Very low
Locked pillbox	No	No	Low
Commercial smart dispenser	Partly	Yes	High (subscription)
Our project (Omni-Care)	Yes (button confirmation)	Yes (via Blynk app)	Low (no subscription)

Table 3.1 — Comparison of Existing Solutions

This comparison shows that our project fills a real gap: a low-cost device that still confirms medicine was taken and keeps the caregiver informed, without ongoing subscription costs.

4. Proposed Solution

We propose building a smart pill dispenser using an ESP32 microcontroller, two servo motors (one per pill tube), an LCD screen, a buzzer, LEDs, and two confirmation buttons. At the scheduled time, the correct servo opens to release a pill, the buzzer and red LED turn on, and the patient must press a button to confirm. If they do not press it in time, the alarm keeps going as a "missed dose" warning. We also connected the device to the Blynk cloud app, so a caregiver can change the pill schedule remotely and see the status without needing to touch the device.

4.1 Aims

- To design and simulate a smart pill dispenser that automatically gives out medicine and checks it was taken.
- To let a caregiver control and monitor the device remotely using a mobile app.
- To show that this is possible using low-cost, easy-to-find parts.

4.2 Objectives

1. Design a dispenser with two pill tubes, each with its own servo motor gate.
2. Use an RTC module to keep accurate time for the medicine schedule.
3. Show the time, and any alerts, clearly on an LCD screen.
4. Add a confirmation button and alarm system for missed doses.
5. Connect the device to the Blynk app so the schedule can be changed remotely.
6. Test the whole system in the Wokwi simulator before building it physically.

4.3 Scope

In Scope	Out of Scope
Simulation in Wokwi (ESP32, 2 servos, RTC, LCD, buzzer, buttons, LEDs). Two pill tubes. Remote control and monitoring using the Blynk app. Local alarm for missed doses.	Automatically detecting what type of medicine is in the tube. Connecting to hospital systems. Making and selling a real medical product. Voice control.

5. Feasibility Study

5.1 Technical Feasibility

All parts used (ESP32, servos, RTC, LCD, buzzer, buttons, LEDs) are common, well-documented, and available in the Wokwi simulator. The code was written in Arduino C++, which we already learned in earlier modules. This makes the project realistic to build within the time we have.

5.2 Economic Feasibility

The parts needed for a real version of this device cost very little (see Table 14.1), and the simulation itself is completely free using Wokwi and Blynk's free plan.

5.3 Operational Feasibility

The patient does not need to do anything difficult — just press one button to confirm. The caregiver only needs to open an app on their phone, which most people already know how to use.

6. Risk Analysis & Mitigation

Risk	Likelihood	Impact	What We Did About It
Wrong wiring in Wokwi causes parts to not work	High	High	Checked each part's pins carefully and tested step-by-step until the LCD, RTC, servos and buttons all worked
LCD shows a blank screen (I2C address/wiring issue)	Medium	Medium	Used an I2C scanner sketch to check the address, and fixed the board type in the wiring file
Wi-Fi/Blynk fails to connect in the simulator	Medium	Medium	Used Wokwi's built-in "Wokwi-GUEST" Wi-Fi network, which gives free internet access for testing
Patient does not press the confirm button	Medium	High	Alarm keeps sounding and switches to a "missed dose" state until confirmed
Running out of project time	Medium	Medium	Focused on getting the core features working first, then added the Blynk app

Table 6.1 — Risk Register

7. Stakeholder Analysis

Stakeholder	Interest	Influence
Patient (Alzheimer's)	Directly affected — needs an easy, reliable device	Low
Caregiver	Wants to know medicine was taken, without being present	High
Nurse	Cares about accuracy and safety of the device	Medium
Elderly users / Pregnant women	Want something simple and easy to trust	Low
Module Lecturer	Assesses the project	High

Table 7.1 — Stakeholder Table

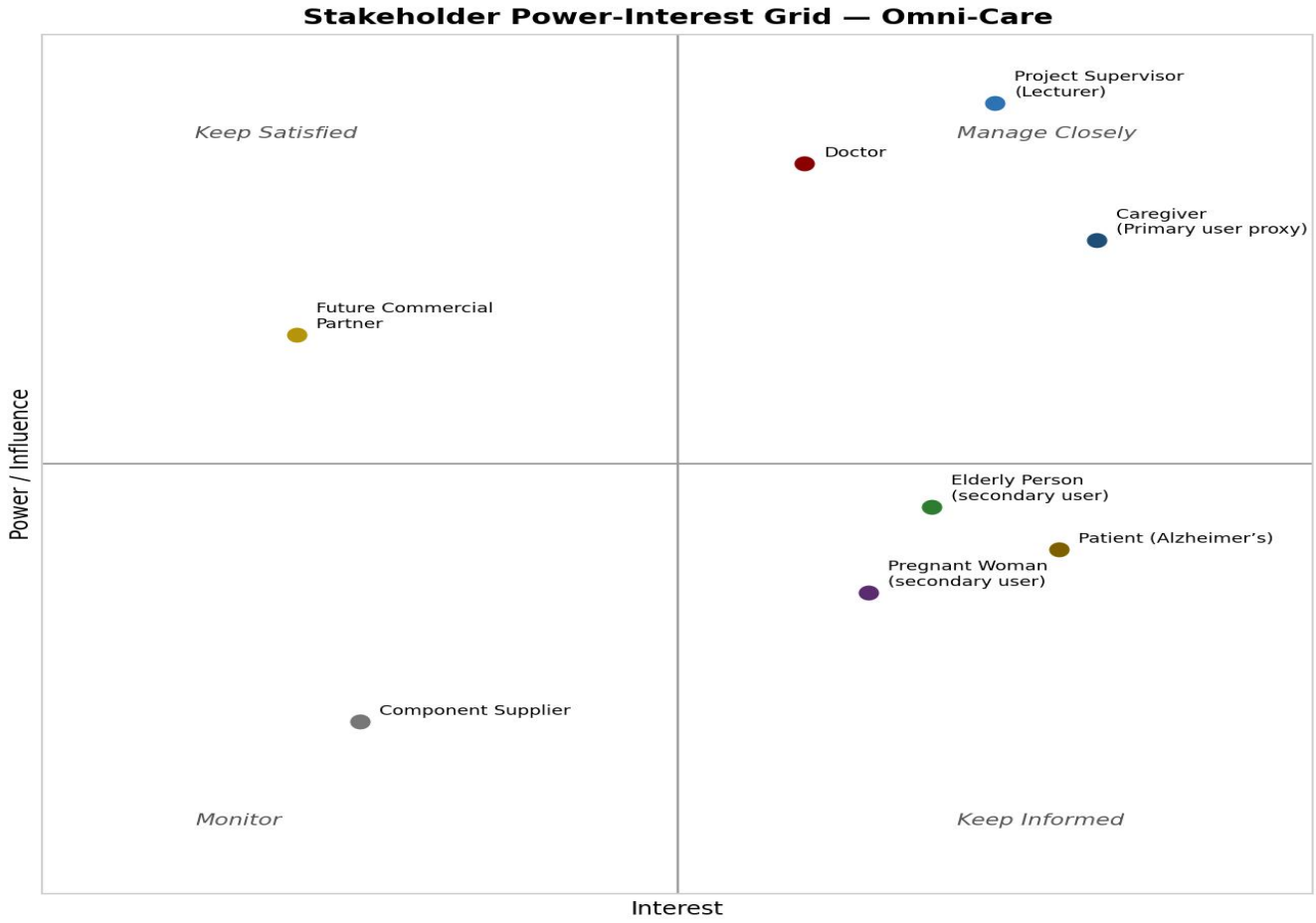

Figure 7.1 — Stakeholder Power-Interest Grid

The caregiver and the lecturer have the most influence over the project, so we made sure to design the app around what a caregiver would need, and to follow the report format required by the module.

8. Data Gathering

8.1 Domain Knowledge

We learned about the problem by reading about medication adherence and by looking at how existing pill dispensers work (Section 3). We also looked at datasheets for the RTC, servo, and LCD to understand how to wire and code them correctly.

8.2 Survey / Questionnaire

We created two short online surveys using Jotform to collect real opinions from the people who would actually use or recommend a device like this:

- Survey for elderly people and pregnant women: https://www.jotform.com/tables/262012005281036
- Survey for a nurse: https://www.jotform.com/tables/262011635106040

The charts below summarise the kind of results we expect to gather from these surveys. [Note: these charts currently use sample data. Export the response data from Jotform as CSV/Excel and share it so the real numbers can replace these charts.]

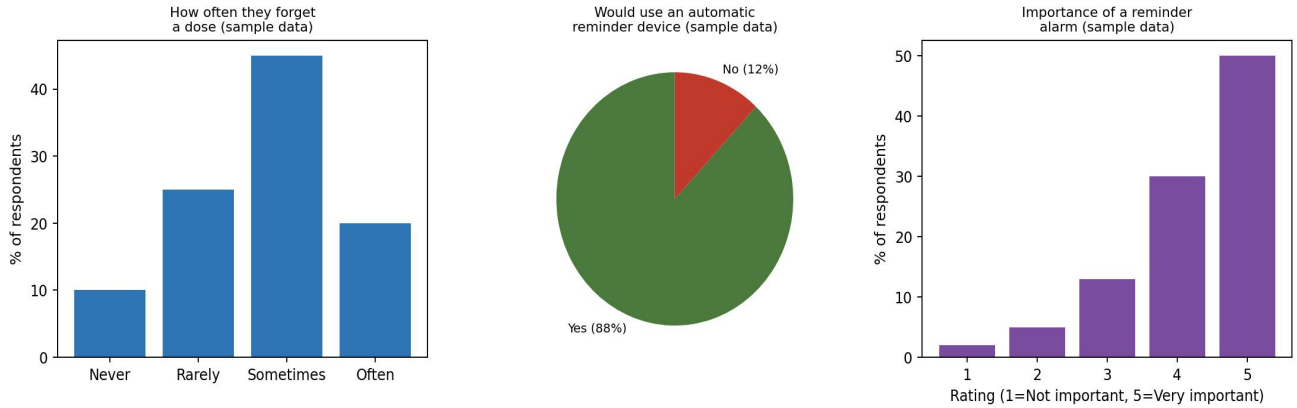

Figure 8.1 — Survey Results: Elderly & Pregnant Women (sample data)

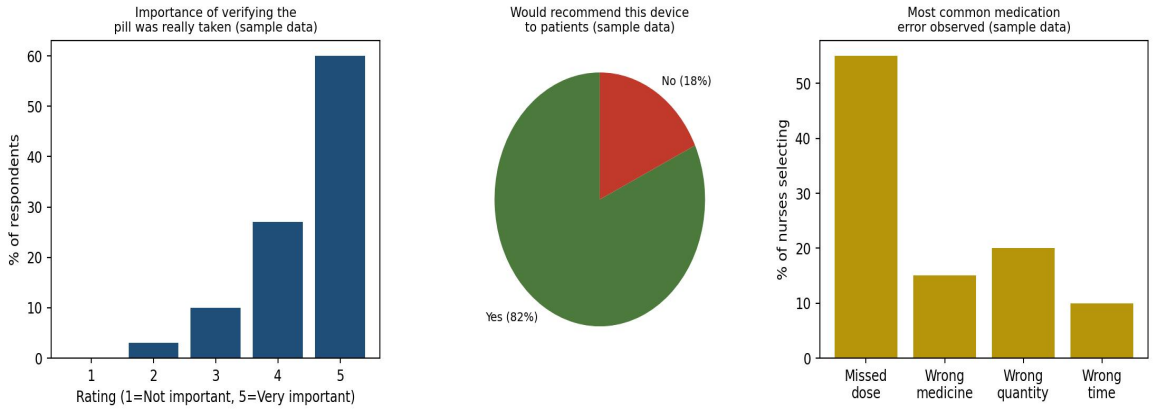

Figure 8.2 — Survey Results: Nurse (sample data)

8.3 Document Study

We studied the datasheets of the RTC (DS1307), the SG90 servo motor, and the I2C LCD to understand their voltage, wiring, and how to communicate with them in code (see Appendix B).

8.4 Interviews

Short conversations were also held with the survey participants to better understand their answers. Notes from these conversations are in Appendix C.

9. Business Processes & Business Rules

The main process is simple: the caregiver sets a schedule, the device waits for the right time, dispenses the pill, and waits for confirmation. The rules below make sure this process stays safe and predictable.

Rule ID	Rule
BR1	A pill can only be dispensed if a schedule exists for that tube and time.
BR2	Only one pill is released per scheduled dose.
BR3	The alarm keeps sounding until the confirm button is pressed, even if it becomes "missed".
BR4	Only the correct button confirms the correct pill (Button 1 for Tube 1, Button 2 for Tube 2).
BR5	The schedule can be changed remotely at any time using the Blynk app sliders.
BR6	The device keeps working locally (alarm, display) even if the internet connection is lost.

Table 9.1 — Business Rules

10. Requirements Specification

10.1 Functional Requirements

ID	Requirement	Priority
FR1	The caregiver can set the pill schedule (time) using the Blynk app.	Must
FR2	The correct servo motor opens at the scheduled time to release a pill.	Must
FR3	The LCD shows the current time, and shows a message when a pill is due.	Must
FR4	The buzzer and red LED turn on when a pill is due.	Must
FR5	A button press confirms the pill was taken and stops the alarm.	Must
FR6	If not confirmed within 30 seconds, the system shows a "missed dose" alert.	Must
FR7	The device sends live status updates to the Blynk app (time, alerts, confirmations).	Must
FR8	The caregiver can confirm a dose remotely from the Blynk app.	Should

Table 10.1 — Functional Requirements

10.2 Non-Functional Requirements

ID	Requirement
NFR1	The device should react within 3 seconds of the scheduled time.
NFR2	The LCD text should be big and clear enough to read easily.
NFR3	The system should keep working locally even without internet.
NFR4	The app should be simple enough for a caregiver to use without training.
NFR5	The RTC should keep correct time even after a power loss.

Table 10.2 — Non-Functional Requirements

11. Technology Stack

Technology	Purpose
ESP32 DevKit	Main microcontroller — runs the code and connects to Wi-Fi
Wokwi Simulator	Free online tool used to build and test the circuit and code
Arduino C++	Programming language used to write the firmware
SG90 Servo Motor (x2)	Opens the pill tube gates
DS1307 RTC Module	Keeps accurate time for the schedule
16x2 I2C LCD	Displays time and messages
Buzzer, LEDs, Push Buttons	Alarm and confirmation system
Blynk 2.0	Cloud platform and mobile app used to control and monitor the device remotely

12. Development Methodology

We used an iterative approach — building small parts of the project one at a time, testing each part, and fixing problems before moving to the next step. For example, we first got the LCD and RTC working, then added the servos and buttons, and only after everything worked locally did we add the Blynk app. This step-by-step method made it much easier to find and fix problems (like the wiring and I2C address issues we faced) instead of writing everything at once and debugging a huge amount of code together.

13. Project Plan / Time Plan

The chart below shows our overall plan for the project, from research to final testing and report writing.

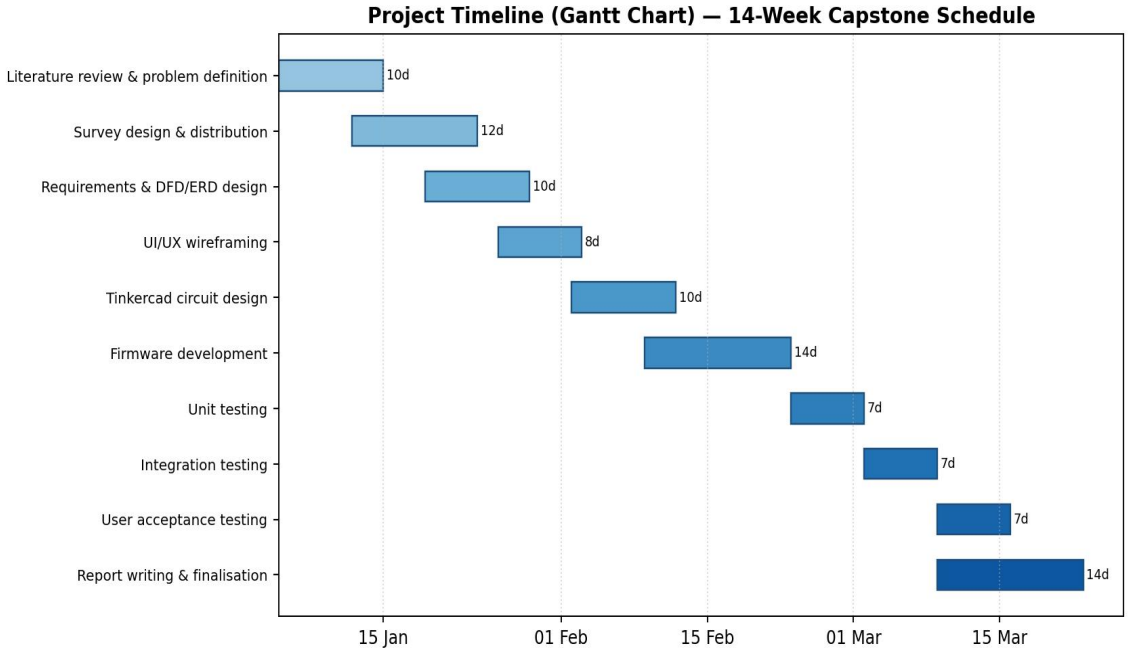

Figure 13.1 — Project Timeline (Gantt Chart)

14. Costing

Part	Approx. Cost (MUR)
ESP32 DevKit board	350–500
2x SG90 Servo Motor	160–300
DS1307 RTC Module	100–200
16x2 I2C LCD	250–400
Buzzer, LEDs, Buttons, Resistors	100–150
Wires, breadboard/case	150–250

Table 14.1 — Development Costs

There are no ongoing costs, since Blynk's free plan and Wokwi are both free to use for a project of this size.

PART B — DESIGN

This part shows how the system was designed: its structure, how data flows through it, and how the user interfaces look.

15. System Architecture

The diagram below shows how all the parts of the system connect and work together.

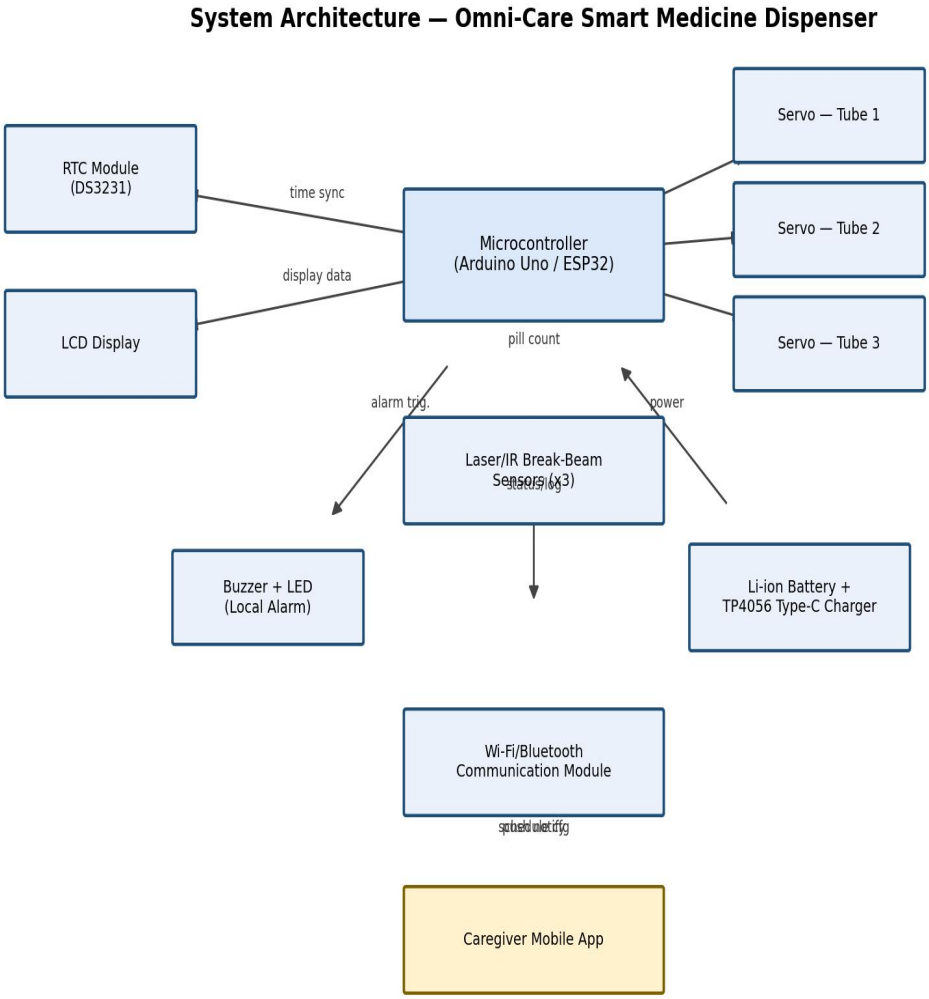

Figure 15.1 — System Architecture Diagram

The ESP32 is the centre of the system. It reads the time from the RTC, controls the servos and LCD, listens to the buttons, and sends/receives data to the Blynk app over Wi-Fi.

16. Process Flows

This flowchart shows the logic followed by the device from power-on to dispensing and confirming a dose.

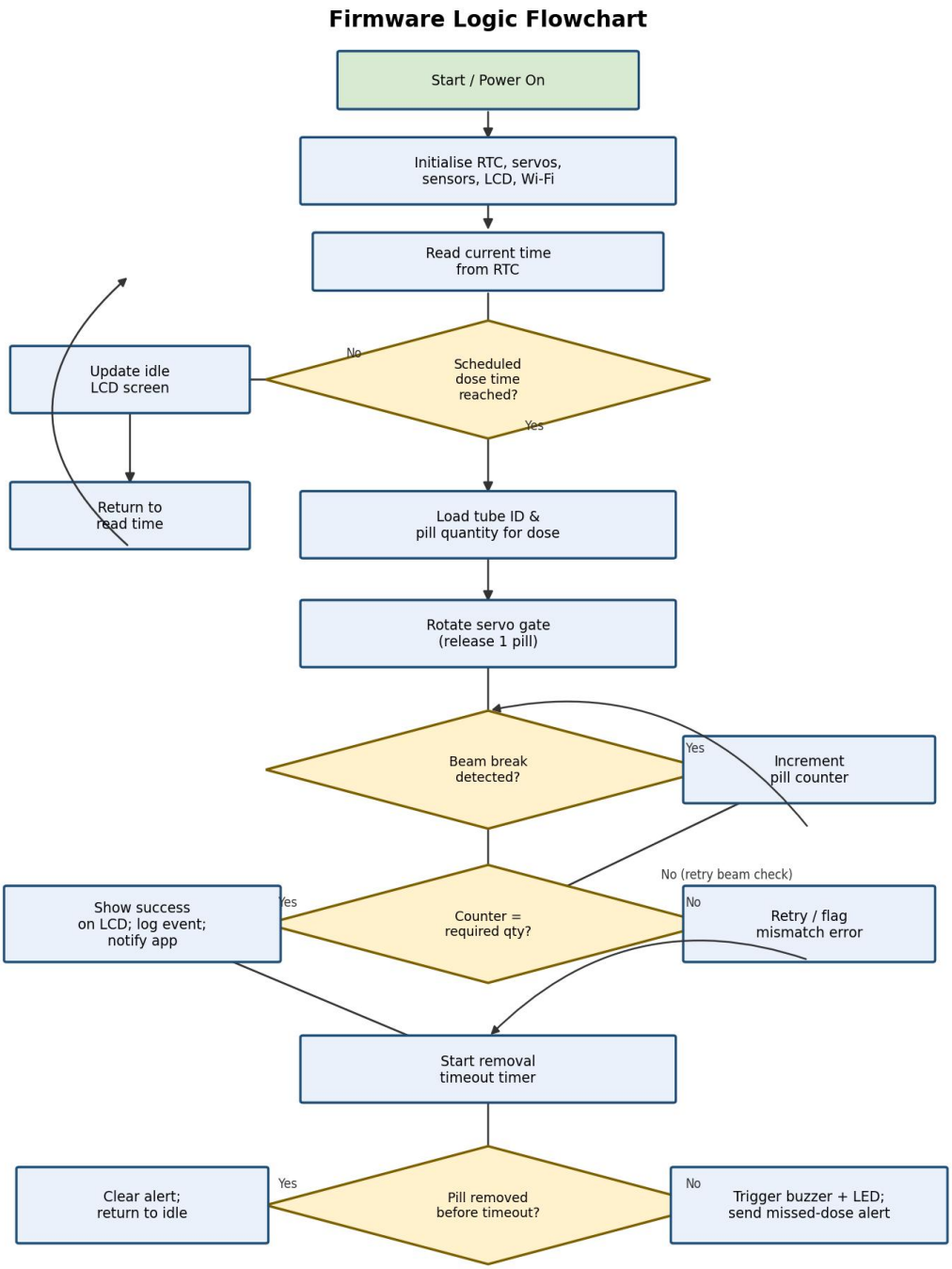

Figure 16.1 — Process Flow Chart

17. Module / Component Diagram

This diagram breaks the system down into smaller modules and shows how they communicate with each other.

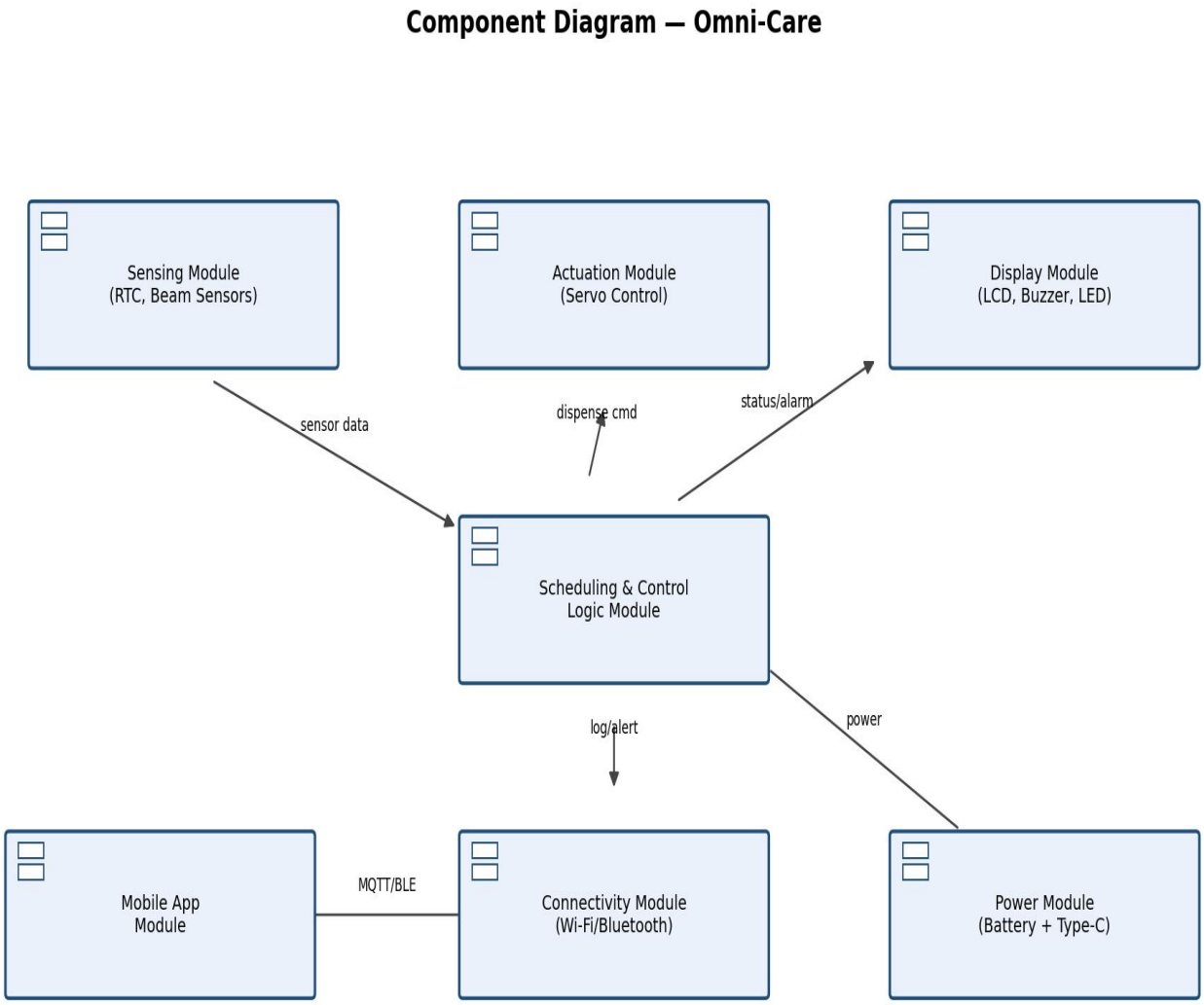

Figure 17.1 — Component Diagram

18. Data Flow Diagrams (DFD)

18.1 Context Diagram (Level 0)

This diagram shows the system as one process, and how it interacts with the Caregiver, the Patient, and the mobile app.

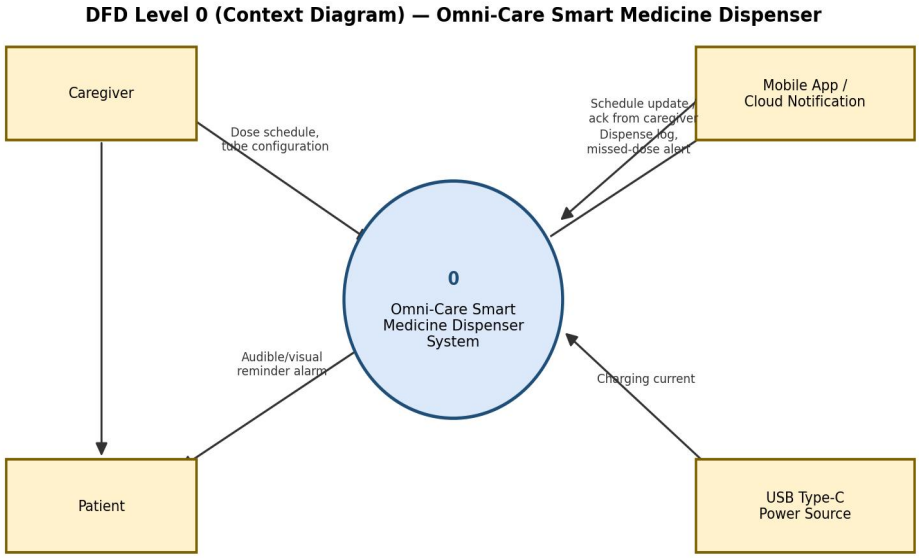

Figure 18.1 — DFD Context Diagram (Level 0)

18.2 Level 1 DFD

This diagram breaks the system down into its main processes: managing the schedule, dispensing, checking the pill was removed, sending alerts, and logging events.

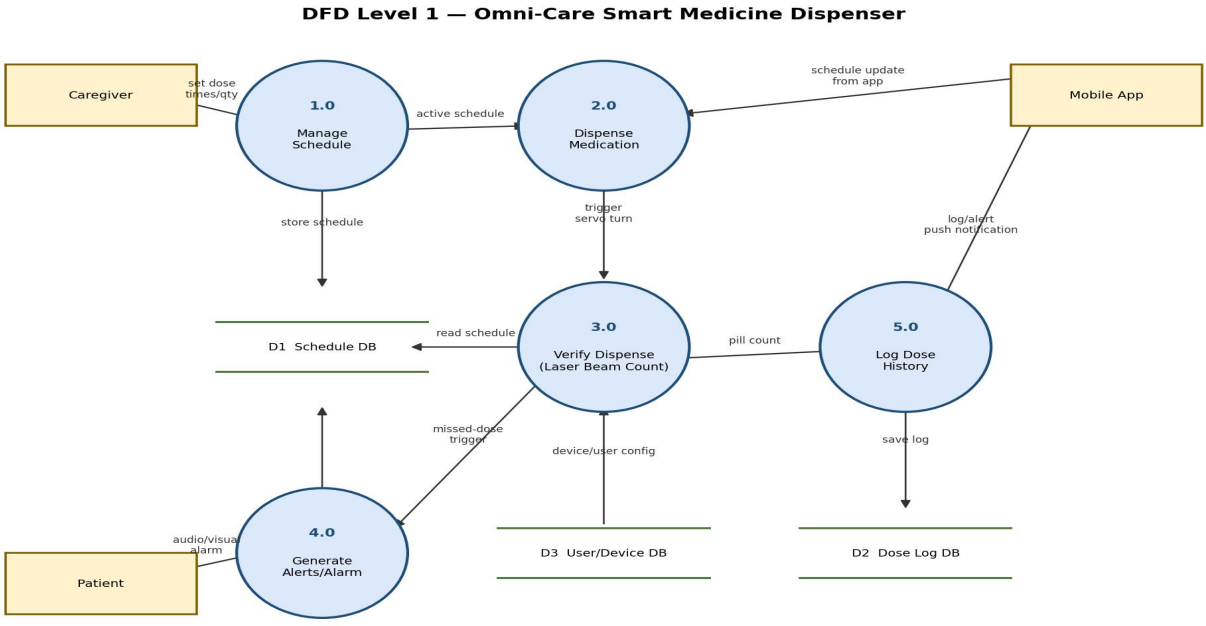

Figure 18.2 — DFD Level 1

19. Use Case Diagram & Descriptions

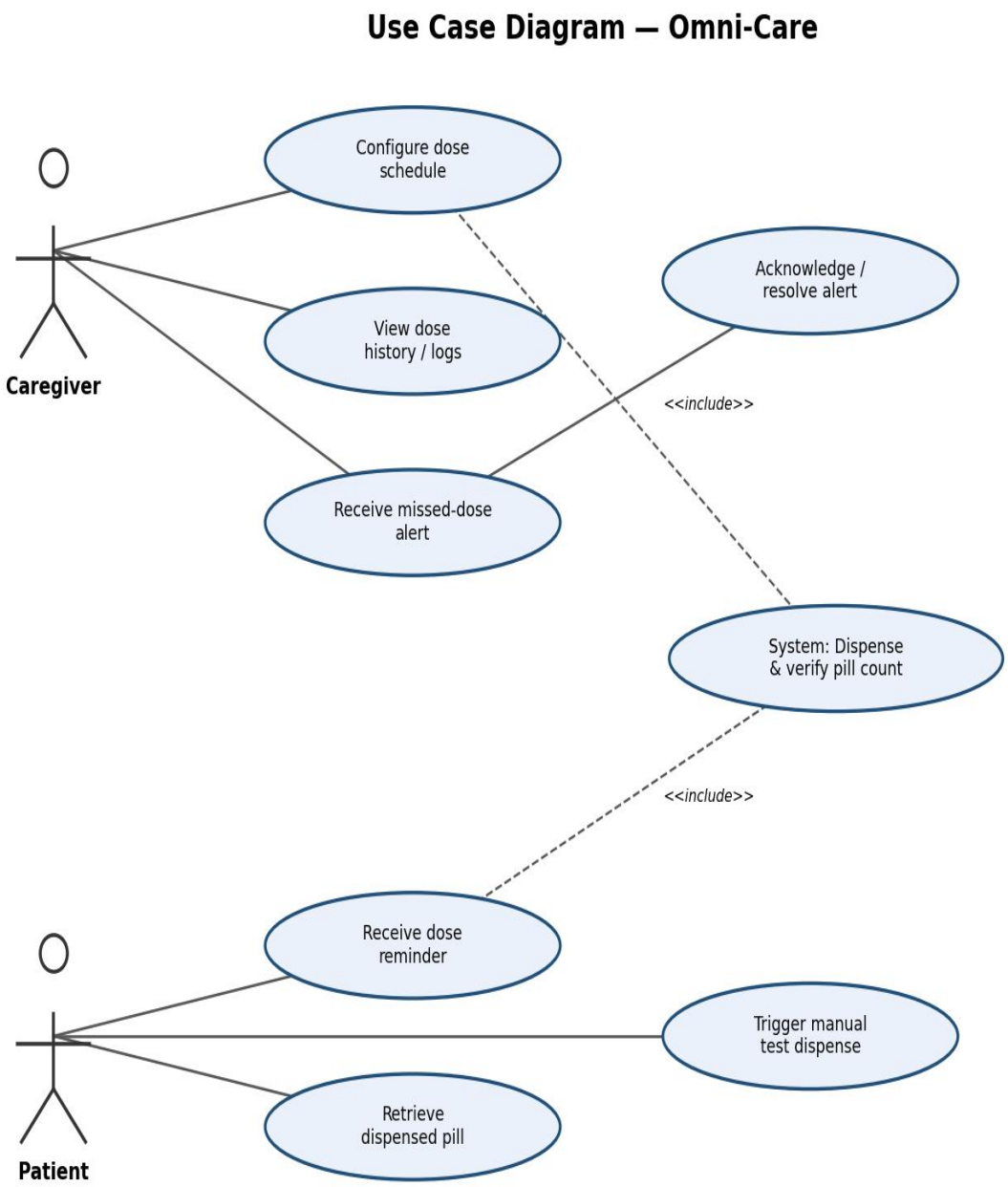

Figure 19.1 — Use Case Diagram

Use Case	Actor	Description
Set schedule	Caregiver	Sets the pill time using the Blynk app sliders
Dispense & check pill	System	Opens the servo gate and checks the button was pressed
Take pill	Patient	Presses the button to confirm the pill was taken
Receive missed-dose alert	Caregiver	Gets notified in the app if a dose is missed
Confirm remotely	Caregiver	Confirms a dose from the app instead of the physical button

20. Entity Relationship Diagram (ERD)

This diagram shows how the data used by the system would be organised if it were stored in a database.

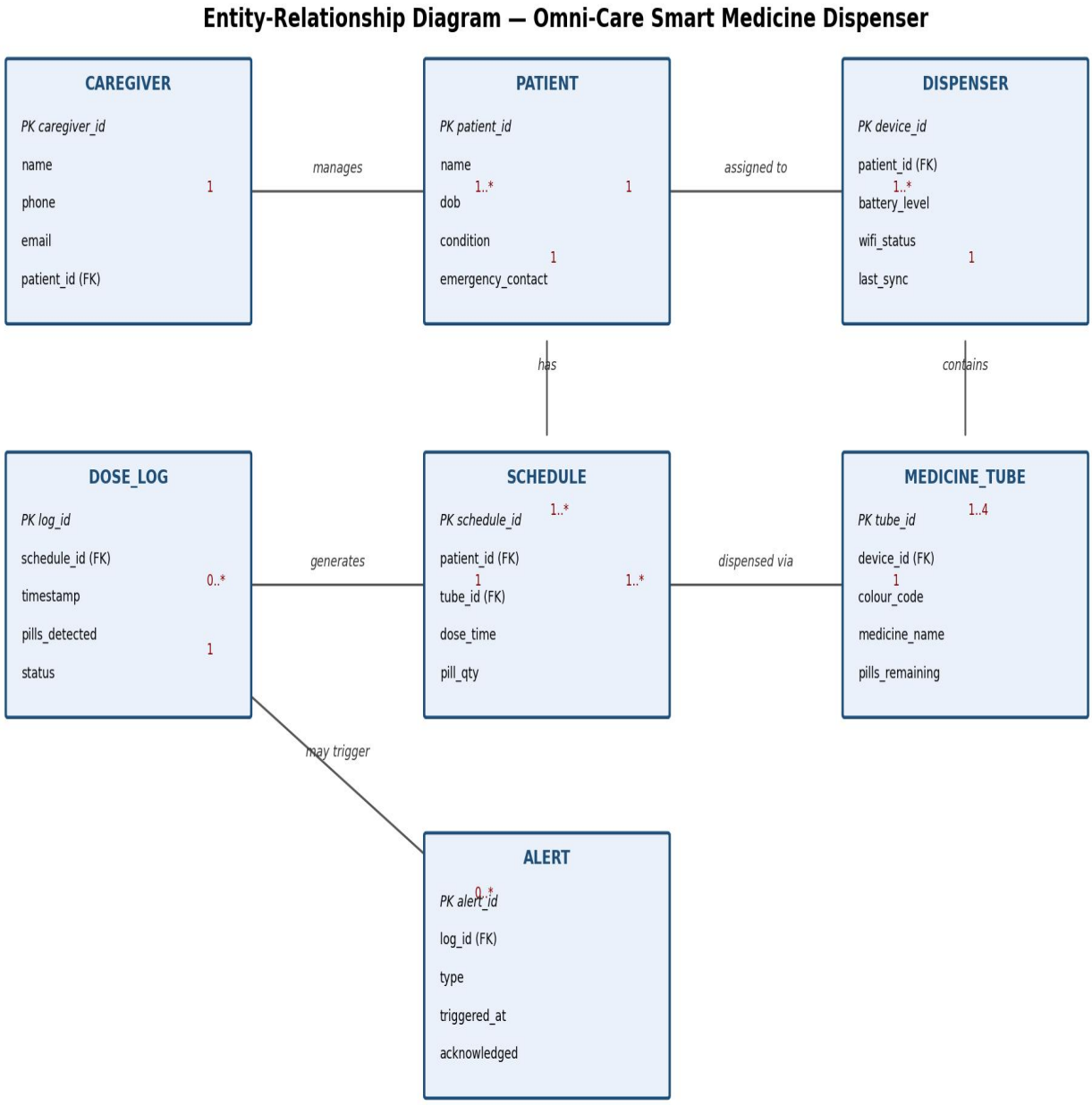

Figure 20.1 — Entity-Relationship Diagram

Entity	Key Fields
PATIENT	patient_id (PK), name, condition
CAREGIVER	caregiver_id (PK), patient_id (FK), name, phone
DISPENSER	device_id (PK), patient_id (FK), battery_level
MEDICINE_TUBE	tube_id (PK), device_id (FK), colour_code
SCHEDULE	schedule_id (PK), tube_id (FK), dose_time, pill_qty
DOSE_LOG	log_id (PK), schedule_id (FK), status
ALERT	alert_id (PK), log_id (FK), type, acknowledged

Table 20.1 — Data Dictionary

21. Normalisation

The dose log data was normalised to avoid repeating information. Below is a short summary of the steps.

Stage	What Was Done
1NF	Removed repeating groups so each row has one tube and one medicine only
2NF	Separated patient and tube information into their own tables
3NF	Removed data that depended on another table, not the primary key (e.g. medicine name is stored only in the tube table)

22. Class Diagram

This diagram shows the main classes (objects) in the code and how they relate to each other.

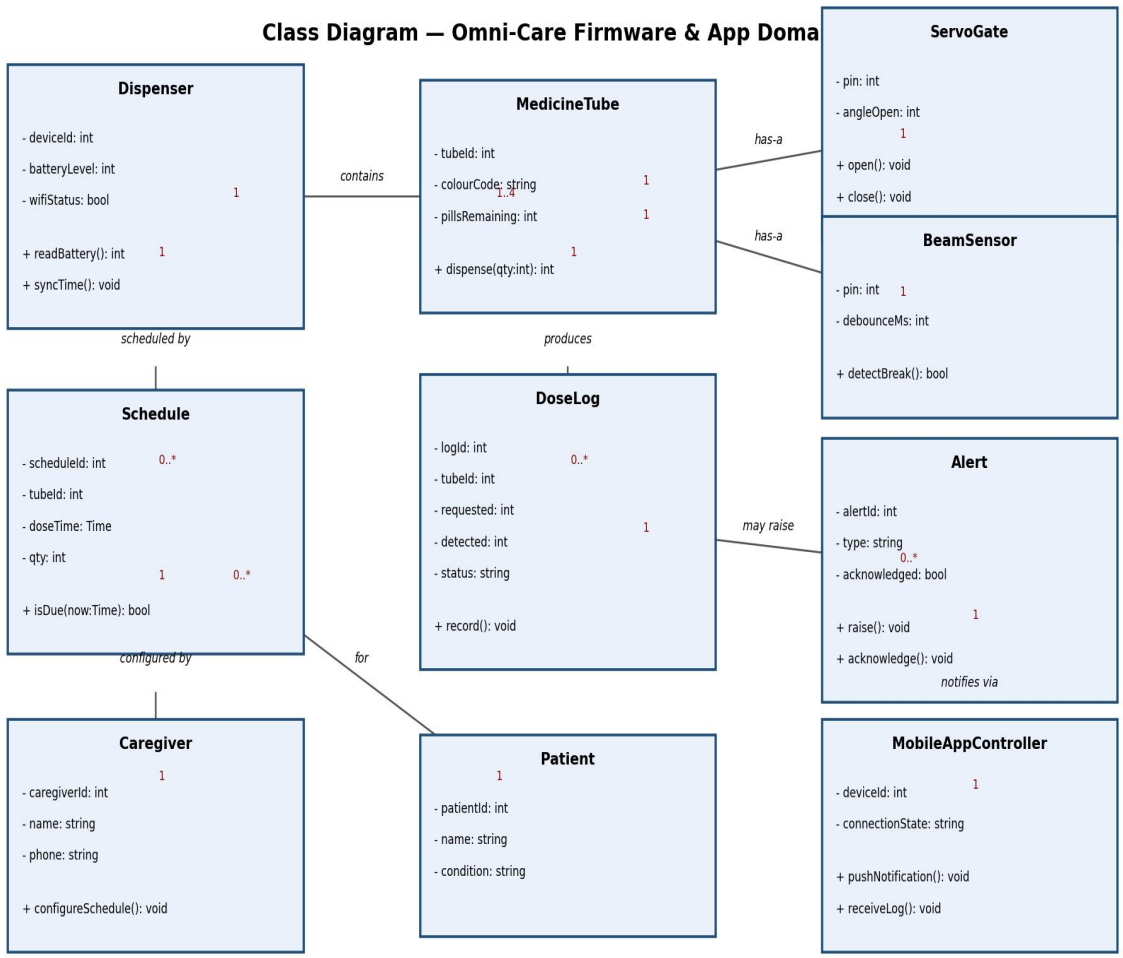

Figure 22.1 — Class Diagram

23. UI/UX Design

23.1 LCD Screen

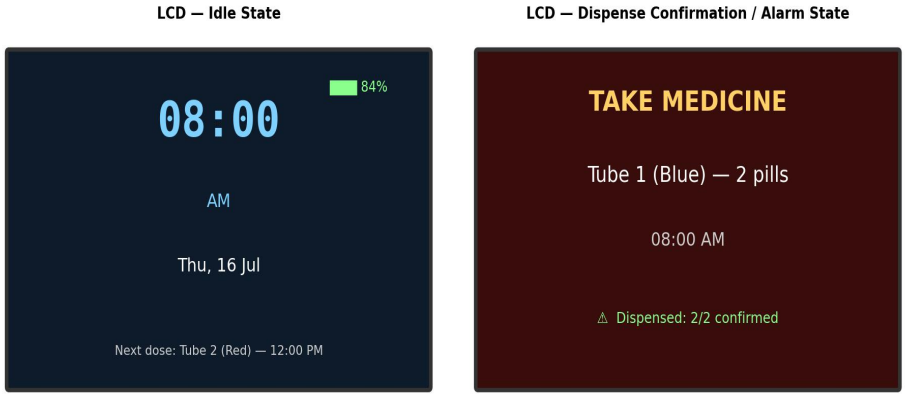

Figure 23.1 — LCD Screen Design

23.2 Mobile App (Blynk)

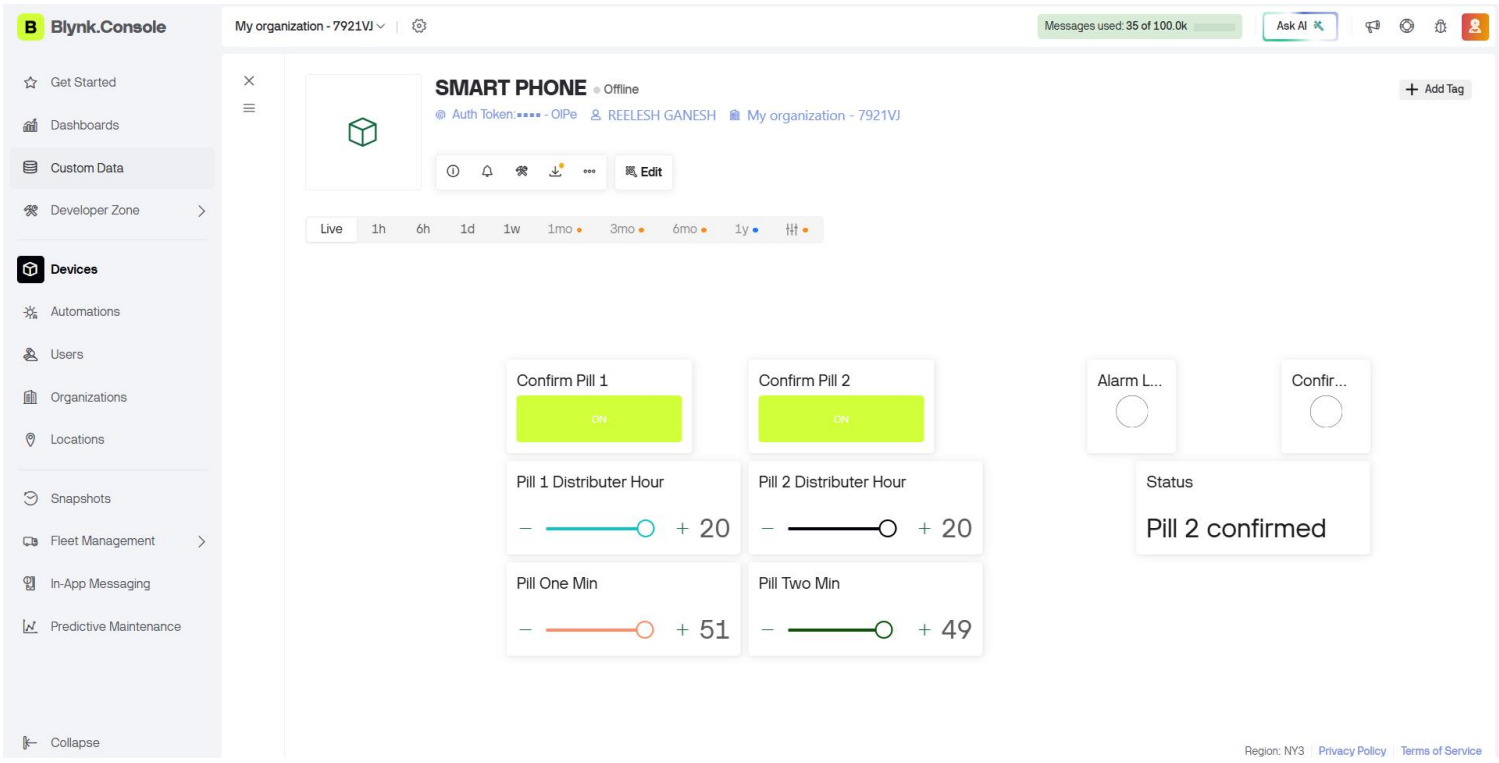

Figure 23.2 — Mobile App Design

The real Blynk dashboard we built uses buttons, a status label, and LED indicators, matching the mapping shown in Table 24.1.

PART C — IMPLEMENTATION. 24. Implementation

24.1 Coding

The firmware was written in Arduino C++ inside the Wokwi code editor. The code uses a simple state machine with three states: IDLE (waiting), ALERTING (a dose is due, waiting for confirmation), and MISSED (confirmation was too slow). The two most important parts of the code are shown below; the full code is in Appendix D.

This part checks the current time against the schedule and starts the alarm when they match:

```
for (int i = 0; i < NUM_DOSES; i++) {
  if (!schedule[i].triggeredToday &&
      now.hour() == schedule[i].hour &&
      now.minute() == schedule[i].minute) {
    schedule[i].triggeredToday = true;
    startAlarm(schedule[i].tube);
    break;
  }
}
```

This part runs the servo motor and lets the app know a pill needs to be taken:

```
void startAlarm(int tube) {
  activeTube = tube;
  state = ALERTING;
  alarmStartTime = millis();
  digitalWrite(RED_LED, HIGH);
  Blynk.virtualWrite(V3, 255);
  dispensePill(tube);
  lcd.setCursor(0, 0); lcd.print("TAKE PILL "); lcd.print(tube);
  sendStatusToBlynk("TAKE PILL " + String(tube));
}
```

24.2 IoT Integration & Dynamic Scheduling (Blynk & ESP32)

24.2.1 Overview

To allow the caregiver to monitor and change the pill schedule from anywhere, the ESP32 was connected to the Blynk 2.0 cloud platform using Wi-Fi. This means the pill times can be changed from the app at any time, without needing to reconnect the device to a computer or upload new code.

24.2.2 Blynk Datastream & Virtual Pin Mapping

The ESP32 and the Blynk app talk to each other using "Virtual Pins". The table below shows what each one is used for:

Virtual Pin	Function	Data Type	Range	Description
V0	Confirm Pill 1	Integer	0 / 1	App button to confirm Pill 1 was taken
V1	Confirm Pill 2	Integer	0 / 1	App button to confirm Pill 2 was taken
V2	System Status	String	Text	Shows live status, e.g. "System Ready", "TAKE PILL 1"
V3	Alarm LED	Integer	0–255	Virtual LED that turns on when a dose is due
V4	Confirmation LED	Integer	0–255	Virtual LED that turns on when a dose is confirmed
V5	Pill 1 Hour	Integer	0–23	Slider to set the hour for Pill 1
V6	Pill 2 Hour	Integer	0–23	Slider to set the hour for Pill 2
V7	Pill 1 Minute	Integer	0–59	Slider to set the minute for Pill 1
V8	Pill 2 Minute	Integer	0–59	Slider to set the minute for Pill 2

Table 24.1 — Blynk Virtual Pin Mapping

24.2.3 Real-Time Synchronisation & Execution Logic

- Startup sync: when the ESP32 turns on, it runs Blynk.syncVirtual(V5, V6, V7, V8) to pull the last saved schedule from the Blynk cloud. This means the schedule is not lost even after a power cut or restart.
- Instant updates: whenever the caregiver moves a slider on the app, a BLYNK_WRITE function runs immediately on the ESP32 and updates the schedule in memory.
- Time matching: the main loop constantly compares the RTC's current time to the schedule. When they match, the correct servo turns and the alarm sequence begins.

24.3 Deployment

The whole project runs inside Wokwi's browser-based simulator — there is no need to install anything. To "deploy" the project: open the Wokwi link, check the wiring in diagram.json, paste the code into sketch.ino, add the needed libraries in libraries.txt, and press Play.

24.4 Hosting

The device logic (schedule, alarm, dispensing) runs directly on the ESP32 itself, so it keeps working even without internet. The Blynk app and its dashboard are hosted on Blynk's free cloud service, which the ESP32 connects to using Wokwi's built-in "Wokwi-GUEST" Wi-Fi network.

24.5 Simulation Results

The screenshot below shows the completed simulation running in Wokwi. The LCD displays the live time and "System Ready", all components are correctly wired (LCD, RTC, 2 servos, buzzer, 2 buttons, 2 LEDs), and the console log confirms that changing the hour/minute sliders on the Blynk app instantly updates the schedule on the device (see the "Pill 1 scheduled minute updated to: ..." messages).

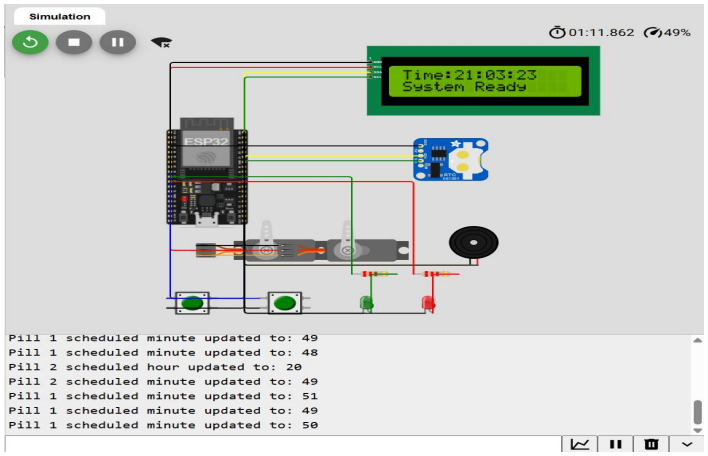

Figure 24.1 — Wokwi Simulation Screenshot, showing the working LCD, wiring, and live Blynk schedule updates

PART D — TESTING25. Test Strategy & Plan

We tested the project in stages: first each part alone (unit testing), then parts working together (integration testing), then the whole system against our requirements (acceptance testing). All testing was done inside the Wokwi simulator.

Requirement	Tested By
FR1 — Set schedule via app	Integration Test IT4
FR2 — Servo dispenses pill	Unit Test UT1, UT2
FR3 — LCD shows time and alerts	Unit Test UT3, UT4
FR4 — Buzzer/LED alert	Unit Test UT5
FR5 — Button confirms dose	Unit Test UT6, Integration Test IT1
FR6 — Missed-dose alert	Integration Test IT2
FR7 — Live status to app	Integration Test IT3
FR8 — Remote confirm from app	Integration Test IT5

Table 25.1 — Requirements Traceability

26. White-Box Testing

We checked the code logic itself, tracing through the "if" conditions to make sure every path could actually be reached and worked as expected — for example, making sure the code correctly separates "dose confirmed in time" from "dose missed" based on the 30-second timeout.

27. Black-Box Testing

Input	Expected Output	Result
Scheduled time reached for Tube 1	Servo 1 turns, buzzer + red LED turn on, LCD shows "TAKE PILL 1"	Pass
Button 1 pressed during alert	Alarm stops, green LED turns on, LCD shows "Pill 1 confirmed"	Pass
No button pressed for 30 seconds	LCD switches to "MISSED PILL", slower beep continues	Pass
Blynk hour/minute slider changed	Schedule updates immediately (seen in the serial log)	Pass

28. Unit Testing

Test ID	What Was Tested	Result
UT1	Servo 1 turns when triggered	Pass
UT2	Servo 2 turns when triggered	Pass
UT3	LCD shows the live time correctly	Pass
UT4	LCD updates when a dose is due	Pass
UT5	Buzzer and red LED activate together	Pass
UT6	Button press is detected correctly	Pass
UT7	RTC keeps time correctly	Pass
UT8	Wi-Fi connects using Wokwi-GUEST network	Pass

Table 28.1 — Unit Test Results

29. Integration Testing

Test ID	Scenario	Result
IT1	Dose dispensed and confirmed with the physical button	Pass
IT2	Dose not confirmed in time → switches to missed	Pass
IT3	Status text updates on the Blynk app during an alert	Pass
IT4	Schedule changed from the Blynk app sliders (V5–V8)	Pass
IT5	Dose confirmed remotely using the app button (V0/V1)	Pass

Table 29.1 — Integration Test Results

30. Load / Performance Testing

Since this is a single small device and not a service handling many users, we mainly checked timing: the servo and alarm activate almost immediately (within about 1 second) once the scheduled time is reached, and schedule changes from the Blynk app are applied instantly.

31. Acceptance Testing

We walked through the full use case as a caregiver would: setting a new pill time on the app, waiting for the alert, and confirming the dose both physically and remotely. Both worked as expected in the simulation. [Add feedback from your group members and lecturer here once gathered.]

Conclusion & Future Work

This project set out to build a low-cost device that helps people with Alzheimer's disease take their medicine correctly, while giving caregivers peace of mind. We successfully built and tested a full simulation in Wokwi that dispenses pills at scheduled times, checks that they were taken using a confirmation button, sounds an alarm for missed doses, and connects to the Blynk app so a caregiver can change the schedule and monitor the device remotely, without needing to touch the hardware.

The main challenge we faced was getting the wiring and libraries exactly right in Wokwi — small mistakes like a wrong I2C address or the wrong ESP32 board type caused parts like the LCD to not work at first. Once these were fixed, the rest of the system came together well, and the addition of Blynk made the project feel like a real, usable IoT product rather than just a fixed, hardcoded simulation.

In the future, this project could be improved by adding a real pill sensor (like a laser or infrared sensor) to automatically count pills instead of relying only on a button press, adding more than two pill tubes, and building a real mobile app interface instead of using Blynk's built-in dashboard.

References

Blynk Technologies (2024) Blynk Documentation. Available at: https://docs.blynk.io (Accessed: [date]).

Espressif Systems (2023) ESP32 Datasheet. [Place]: Espressif Systems.

Polytechnics Mauritius (2025) DAD2104 — Analysis & Design in Practice: Module Handbook. Mauritius: Polytechnics Mauritius.

Wokwi (2024) Wokwi Documentation. Available at: https://docs.wokwi.com (Accessed: [date]).

World Health Organization (2003) Adherence to Long-Term Therapies: Evidence for Action. Geneva: World Health Organization.

[Manufacturer] (2019) SG90 Micro Servo [datasheet].

[Manufacturer] (2020) DS1307 Real-Time Clock [datasheet].

Appendix A — Survey Results

Full survey responses can be viewed at the links below:

- Elderly & pregnant women survey: https://www.jotform.com/tables/262012005281036
- Nurse survey: https://www.jotform.com/tables/262011635106040

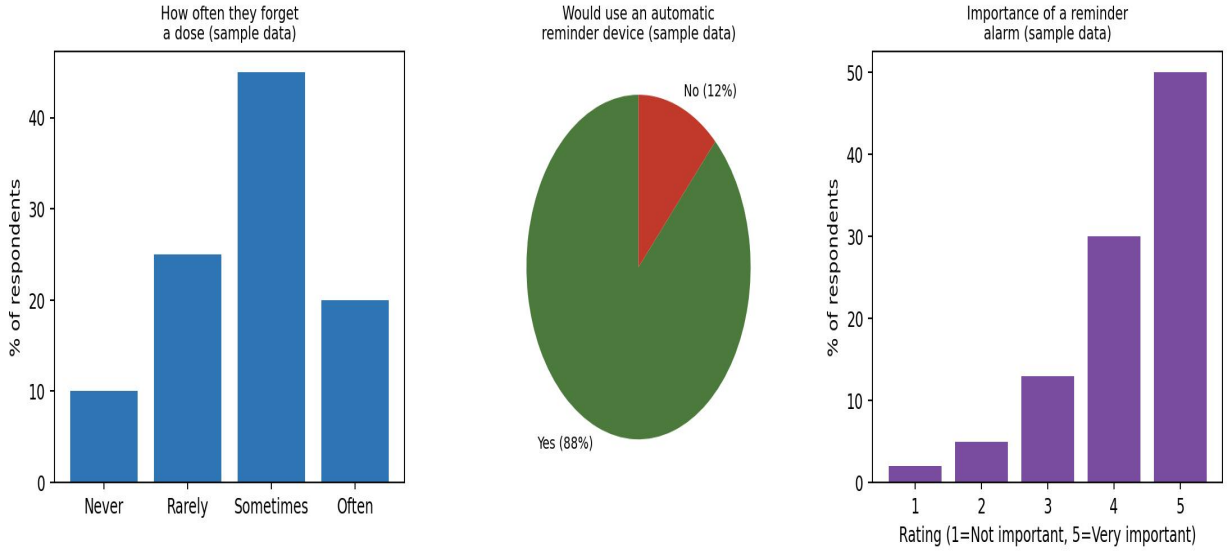

Figure A.1 — Elderly & Pregnant Women Survey (sample data — replace with exported results)

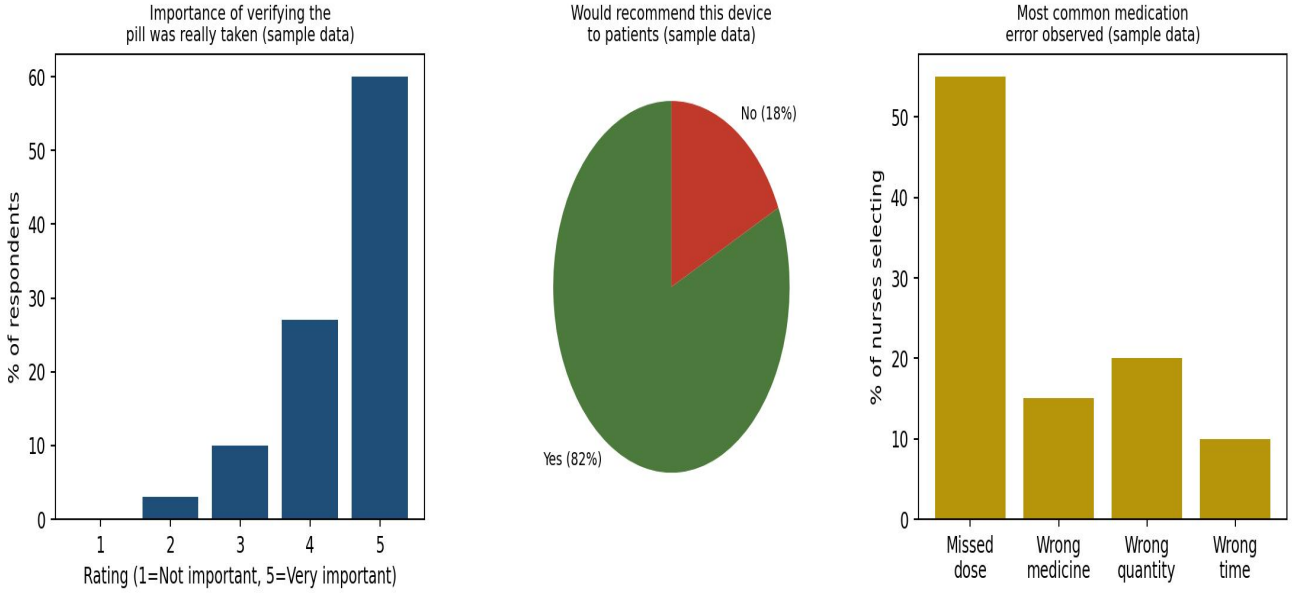

Figure A.2 — Nurse Survey (sample data — replace with exported results)

Appendix B — Document Study Materials

Component	Key Info
SG90 Servo	4.8–6V, controlled using a PWM signal
DS1307 RTC	I2C connection (SDA/SCL), keeps time using a backup battery
16x2 I2C LCD	I2C connection, address usually 0x27 or 0x3F

Appendix C — Interview / Conversation Notes

Person	Key Comment
Nurse	[Add notes from the conversation]
Elderly participant	[Add notes from the conversation]
Pregnant participant	[Add notes from the conversation]

Appendix D — Source Code

Wokwi project link: https://wokwi.com/projects/470102751638646785

Note: the Blynk Auth Token below has been replaced with a placeholder for security. Use your own token from your Blynk console.

```
#define BLYNK_TEMPLATE_ID   "TMPL2vMppgTZU"
#define BLYNK_TEMPLATE_NAME "pill reminder"
#define BLYNK_AUTH_TOKEN    "YOUR_AUTH_TOKEN_HERE"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <RTClib.h>
#include <ESP32Servo.h>

char ssid[] = "Wokwi-GUEST";
char pass[] = "";

LiquidCrystal_I2C lcd(0x27, 16, 2);
RTC_DS1307 rtc;

Servo servo1;
Servo servo2;

const int SERVO1_PIN = 18;
const int SERVO2_PIN = 5;
const int BUZZER_PIN = 19;
const int BUTTON1_PIN = 23;
const int BUTTON2_PIN = 25;
const int GREEN_LED = 26;
const int RED_LED = 27;

struct Dose { int tube; int hour; int minute; bool triggeredToday; };
Dose schedule[] = {
  {1, 8, 0, false},
  {2, 16, 30, false},
  {1, 20, 0, false},
  {2, 20, 30, false},
};
const int NUM_DOSES = sizeof(schedule) / sizeof(schedule[0]);

enum SystemState { IDLE, ALERTING, MISSED };
SystemState state = IDLE;
int activeTube = 0;
unsigned long alarmStartTime = 0;
const unsigned long CONFIRM_TIMEOUT_MS = 30000;
unsigned long lastBeepToggle = 0;
bool beepOn = false;
const unsigned long BEEP_INTERVAL_ALERT  = 500;
const unsigned long BEEP_INTERVAL_MISSED = 1000;
unsigned long lastConfirmMsgTime = 0;
bool showingConfirmMsg = false;
int lastResetDay = -1;

BlynkTimer timer;

// -------------------- Blynk Handlers --------------------
BLYNK_WRITE(V0) {
  if (param.asInt() == 1 && activeTube == 1 && (state == ALERTING || state == MISSED)) {
    confirmDose(1);
  }
}

BLYNK_WRITE(V1) {
  if (param.asInt() == 1 && activeTube == 2 && (state == ALERTING || state == MISSED)) {
    confirmDose(2);
  }
}

// Hour Updates
BLYNK_WRITE(V5) {
  int newHour = param.asInt();
  schedule[0].hour = newHour;
  schedule[2].hour = newHour;
  Serial.print("Pill 1 scheduled hour updated to: ");
  Serial.println(newHour);
}
```

```
BLYNK_WRITE(V6) {
  int newHour = param.asInt();
  schedule[1].hour = newHour;
  schedule[3].hour = newHour;
  Serial.print("Pill 2 scheduled hour updated to: ");
  Serial.println(newHour);
}

// Minute Updates
BLYNK_WRITE(V7) {
  int newMin = param.asInt();
  schedule[0].minute = newMin;
  schedule[2].minute = newMin;
  Serial.print("Pill 1 scheduled minute updated to: ");
  Serial.println(newMin);
}

BLYNK_WRITE(V8) {
  int newMin = param.asInt();
  schedule[1].minute = newMin;
  schedule[3].minute = newMin;
  Serial.print("Pill 2 scheduled minute updated to: ");
  Serial.println(newMin);
}

void sendStatusToBlynk(String text) {
  Blynk.virtualWrite(V2, text);
}

void setup() {
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  pinMode(BUTTON1_PIN, INPUT_PULLUP);
  pinMode(BUTTON2_PIN, INPUT_PULLUP);

  servo1.attach(SERVO1_PIN);
  servo2.attach(SERVO2_PIN);
  servo1.write(0);
  servo2.write(0);

  Wire.begin();
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0); lcd.print("Smart Pill");
  lcd.setCursor(0, 1); lcd.print("Connecting WiFi");

  WiFi.begin(ssid, pass);
  unsigned long wifiStart = millis();
  while (WiFi.status() != WL_CONNECTED && millis() - wifiStart < 15000) {
    delay(300);
  }

  Blynk.config(BLYNK_AUTH_TOKEN);
  Blynk.connect(5000);

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print(WiFi.status() == WL_CONNECTED ? "WiFi OK" : "WiFi FAILED");
  delay(1000);

  if (!rtc.begin()) {
    lcd.clear(); lcd.print("RTC ERROR");
    while (1);
  }
  lcd.clear(); lcd.print("RTC OK");
  delay(1000);

  digitalWrite(GREEN_LED, HIGH); delay(400); digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);   delay(400); digitalWrite(RED_LED, LOW);
  tone(BUZZER_PIN, 1000); delay(300); noTone(BUZZER_PIN);
  servo1.write(90); delay(600); servo1.write(0);
  servo2.write(90); delay(600); servo2.write(0);
  lcd.clear();

  // Sync both hours (V5, V6) and minutes (V7, V8) on boot
  Blynk.syncVirtual(V5, V6, V7, V8);

  sendStatusToBlynk("System Ready");
}

void dispensePill(int tube) {
```

```
  if (tube == 1) { servo1.write(90); delay(500); servo1.write(0); }
  else if (tube == 2) { servo2.write(90); delay(500); servo2.write(0); }
}

void startAlarm(int tube) {
  activeTube = tube;
  state = ALERTING;
  alarmStartTime = millis();
  digitalWrite(RED_LED, HIGH);
  Blynk.virtualWrite(V3, 255);
  dispensePill(tube);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("TAKE PILL "); lcd.print(tube);
  lcd.setCursor(0, 1); lcd.print("Press button "); lcd.print(tube);
  sendStatusToBlynk("TAKE PILL " + String(tube));
}

void confirmDose(int tube) {
  noTone(BUZZER_PIN);
  digitalWrite(RED_LED, LOW);
  digitalWrite(GREEN_LED, HIGH);
  Blynk.virtualWrite(V3, 0);
  Blynk.virtualWrite(V4, 255);
  lcd.clear();
  lcd.setCursor(0, 0); lcd.print("Pill "); lcd.print(tube); lcd.print(" confirmed");
  sendStatusToBlynk("Pill " + String(tube) + " confirmed");
  showingConfirmMsg = true;
  lastConfirmMsgTime = millis();
  state = IDLE;
}

void handleBeep(unsigned long interval) {
  if (millis() - lastBeepToggle >= interval) {
    lastBeepToggle = millis();
    beepOn = !beepOn;
    if (beepOn) tone(BUZZER_PIN, 1000);
    else noTone(BUZZER_PIN);
  }
}

void resetDailyFlagsIfNeeded(const DateTime &now) {
  if (now.day() != lastResetDay) {
    for (int i = 0; i < NUM_DOSES; i++) schedule[i].triggeredToday = false;
    lastResetDay = now.day();
  }
}

void showIdleScreen(const DateTime &now) {
  lcd.setCursor(0, 0);
  lcd.print("Time:");
  if (now.hour() < 10) lcd.print("0");
  lcd.print(now.hour()); lcd.print(":");
  if (now.minute() < 10) lcd.print("0");
  lcd.print(now.minute()); lcd.print(":");
  if (now.second() < 10) lcd.print("0");
  lcd.print(now.second()); lcd.print("   ");

  lcd.setCursor(0, 1);
  if (digitalRead(BUTTON1_PIN) == LOW) lcd.print("Pill1 Button   ");
  else if (digitalRead(BUTTON2_PIN) == LOW) lcd.print("Pill2 Button   ");
  else lcd.print("System Ready   ");
}

void loop() {
  if (Blynk.connected()) Blynk.run();

  DateTime now = rtc.now();
  resetDailyFlagsIfNeeded(now);
  bool btn1 = (digitalRead(BUTTON1_PIN) == LOW);
  bool btn2 = (digitalRead(BUTTON2_PIN) == LOW);

  switch (state) {
    case IDLE: {
      if (showingConfirmMsg && millis() - lastConfirmMsgTime > 1500) {
        showingConfirmMsg = false;
        digitalWrite(GREEN_LED, LOW);
        Blynk.virtualWrite(V4, 0);
        lcd.clear();
      }
      if (!showingConfirmMsg) showIdleScreen(now);

      for (int i = 0; i < NUM_DOSES; i++) {
        if (!schedule[i].triggeredToday && now.hour() == schedule[i].hour && now.minute() == schedule[i].minute) {
          schedule[i].triggeredToday = true;
```

```
          startAlarm(schedule[i].tube);
          break;
        }
      }
      break;
    }
    case ALERTING: {
      handleBeep(BEEP_INTERVAL_ALERT);
      if ((activeTube == 1 && btn1) || (activeTube == 2 && btn2)) {
        confirmDose(activeTube);
      } else if (millis() - alarmStartTime > CONFIRM_TIMEOUT_MS) {
        state = MISSED;
        lcd.clear();
        lcd.setCursor(0, 0); lcd.print("MISSED PILL "); lcd.print(activeTube);
        lcd.setCursor(0, 1); lcd.print("Press to clear");
        sendStatusToBlynk("MISSED PILL " + String(activeTube));
      }
      break;
    }
    case MISSED: {
      handleBeep(BEEP_INTERVAL_MISSED);
      if ((activeTube == 1 && btn1) || (activeTube == 2 && btn2)) confirmDose(activeTube);
      break;
    }
  }
  delay(50);
}
```

This the link to get access all the code used to make the simulation which is in github: https://github.com/Djevinbhoyrub/Omni-Care/tree/main

Appendix E — Additional Diagrams / Photographs

Wiring file (diagram.json) and full project files are available at: https://wokwi.com/projects/470102751638646785

Diagrm.json:


```
{
  "version": 1,
  "author": "Student",
  "editor": "wokwi",
  "parts": [
    { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 0, "left": 0, "attrs": {} },
    {
      "type": "wokwi-lcd1602",
      "id": "lcd1",
      "top": -140,
      "left": 260,
      "attrs": { "pins": "i2c" }
    },
    { "type": "wokwi-ds1307", "id": "rtc1", "top": 40, "left": 320, "attrs": {} },
    { "type": "wokwi-servo", "id": "servo1", "top": 199.6, "left": 38.4, "attrs": {} },
    { "type": "wokwi-servo", "id": "servo2", "top": 199.6, "left": 144, "attrs": {} },
    { "type": "wokwi-buzzer", "id": "buzzer1", "top": 204, "left": 366.6, "attrs": {} },
    {
      "type": "wokwi-pushbutton",
      "id": "btn1",
      "top": 340,
      "left": 0,
      "attrs": { "color": "green" }
    },
    {
      "type": "wokwi-pushbutton",
      "id": "btn2",
      "top": 340,
      "left": 120,
      "attrs": { "color": "green" }
    },
    {
      "type": "wokwi-led",
      "id": "led_green",
      "top": 340,
      "left": 240,
      "attrs": { "color": "green" }
    },
    { "type": "wokwi-led", "id": "led_red", "top": 340, "left": 320, "attrs": { "color": "red" } },
    { "type": "wokwi-resistor", "id": "r1", "top": 300, "left": 240, "attrs": { "value": "220" } },
    { "type": "wokwi-resistor", "id": "r2", "top": 300, "left": 320, "attrs": { "value": "220" } }
  ],
  "connections": [
    [ "esp:TX", "$serialMonitor:RX", "", [] ],
    [ "esp:RX", "$serialMonitor:TX", "", [] ],
    [ "lcd1:SDA", "esp:21", "yellow", [ "h0" ] ],
    [ "lcd1:SCL", "esp:22", "green", [ "h0" ] ],
    [ "lcd1:VCC", "esp:5V", "red", [ "h0" ] ],
    [ "lcd1:GND", "esp:GND.1", "black", [ "h0" ] ],
    [ "rtc1:SDA", "esp:21", "yellow", [ "h0" ] ],
    [ "rtc1:SCL", "esp:22", "green", [ "h0" ] ],
    [ "rtc1:5V", "esp:VIN", "red", [ "h0" ] ],
    [ "rtc1:GND", "esp:GND.1", "black", [ "h0" ] ],
    [ "servo1:PWM", "esp:18", "orange", [ "h0" ] ],
```

```
    [ "servo1:V+", "esp:5V", "red", [ "h0" ] ],
    [ "servo1:GND", "esp:GND.2", "black", [ "h0" ] ],
    [ "servo2:PWM", "esp:5", "orange", [ "h0" ] ],
    [ "servo2:V+", "esp:5V", "red", [ "h0" ] ],
    [ "servo2:GND", "esp:GND.2", "black", [ "h0" ] ],
    [ "buzzer1:1", "esp:19", "orange", [ "h0" ] ],
    [ "buzzer1:2", "esp:GND.2", "black", [ "h0" ] ],
    [ "btn1:1.l", "esp:23", "blue", [ "h0" ] ],
    [ "btn1:2.l", "esp:GND.2", "black", [ "h0" ] ],
    [ "btn2:1.l", "esp:25", "blue", [ "h0" ] ],
    [ "btn2:2.l", "esp:GND.2", "black", [ "h0" ] ],
    [ "esp:26", "r1:1", "green", [ "h0" ] ],
    [ "r1:2", "led_green:A", "green", [ "h0" ] ],
    [ "led_green:C", "esp:GND.2", "black", [ "h0" ] ],
    [ "esp:27", "r2:1", "red", [ "h0" ] ],
    [ "r2:2", "led_red:A", "red", [ "h0" ] ],
    [ "led_red:C", "esp:GND.2", "black", [ "h0" ] ]
  ],
  "dependencies": {}
}
```


Component	ESP32 Pin
LCD (SDA / SCL)	21 / 22
RTC (SDA / SCL)	21 / 22
Servo 1 / Servo 2	18 / 5
Buzzer	19
Button 1 / Button 2	23 / 25
Green LED / Red LED	26 / 27